Linux Threads

```
#include <pthread.h> // POSIX threads library
#include <stdio.h>
#include <unistd.h>  // For sleep() and getpid() functions

/* Global variable to store the thread ID.
   Shared across all threads in this process. */
pthread_t my_thread;



/* Helper function to display the IDs of the calling thread */
void print_my_ids(void)
{
    pid_t pid;      // Process ID (Common to all threads in the
process)
    pthread_t tid;  // Thread ID (Unique identifier within the
pthread library)

    pid = getpid();       // System call to get the Process ID
from the Kernel
    tid = pthread_self(); // Library function to get the current
thread's ID

    printf("My PID = %u, My TID = %u\n", (unsigned int)pid,
(unsigned int)tid);
}



/* Entry point function for the newly created thread */
void* foo(void* arg)
{
    /* Print the ID stored in the global variable upon creation
*/
    printf("Thread:%u\n", (unsigned int)my_thread);

    /* Call helper to show this is a separate thread in the same
process */
    print_my_ids();

    return NULL; // Returning from this function terminates the
thread
}
```


© יצחק נודלר

```
int main(int argc, const char* argv[])
{
    /* pthread_create: Spawns a new thread executing the 'foo'
function.
       Returns 0 on success, or an error code otherwise. */
    if (pthread_create(&my_thread, NULL, foo, NULL)) {
        return 1;
    }

    /* Execution inside the Main Thread */
    printf("Main Thread:");
    print_my_ids();

    /* * Pause for 2 seconds. This is crucial to prevent the Main
Thread
     * from finishing main(). If main() exits, the Kernel
terminates
     * the entire process and all its threads.
     */
    sleep(2);

    return 0;
}
```

הסבר מפורט (System Programming Perspective)

חשוב להבין ש-Linux רואה חוטים (Threads) כ- "תהליכים קלי משקל" (**Lightweight Processes**). הם חולקים את אותו מרחב זיכרון אך פועלים כיישיות ביצוע נפרדות.

טיפוס הנתונים `pthread_t`

הטיפוס `pthread_t` הוא הדרך של התוכנית שלנו לזהות חוט ספציפי.

מה הם הערכים שלו? ב-Linux, מדובר בדרך כלל ב-`unsigned long`. זהו אינו מספר אקראי; ברוב המימושים (glibc), הערך הזה הוא למעשה **כתובת בזיכרון** המצביעה על מבנה נתונים (Thread Control Block) בתוך מרחב הכתובות של המשתמש.

השדות שלו: כמתכנתים, אנו מתייחסים אליו כאל "Opaque Type" (טיפוס אטום). אנחנו לא ניגשים לשדות שלו ישירות, אלא משתמשים בפונקציות הספרייה. הוא מכיל מידע על המחסנית (Stack) של החוט, מצב הריצה שלו וערכי החזרה.

© יצחק נודלר

מנקודת מבטו של ה-Kernel: זהו פרט קריטי למתכנתי מערכות – ה- Kernel לא באמת משתמש ב-`pthread_t`. ה-Kernel מנהל חוטים באמצעות מזהה פנימי שנקרא **TID**. ה-`pthread_t` קיים רק בתוך "ספריית המשתמש" (User Space) כדי לאפשר לספריית ה-pthreads לנהל את החוטים מבלי להעיק על ה-Kernel בכל פעולה קטנה.

קריאת המערכת pthread_create()

זוהי פונקציית המעטפת (Wrapper) לקריאת המערכת המורכבת `clone()`.

<u>הארגומנטים:</u>

1. **`&my_thread`:** מצביע למשתנה שבו הספרייה תכתוב את המזהה של החוט החדש שנוצר.
2. **`attr` (`NULL`):** מאפשר להגדיר מאפיינים מיוחדים (כמו גודל מחסנית).
3. `NULL` אומר ל-Linux להשתמש בברירת המחדל (לרוב 8MB).
4. **`start_routine` (`foo`):** מצביע לפונקציה. זהו ה-"Main" של החוט החדש. שם הוא מתחיל את חייו.
5. **`arg` (`NULL`):** כאן ניתן להעביר נתונים לתוך החוט. מאחר והטיפוס הוא `void*`, ניתן להעביר מצביע לכל דבר.

<u>ערך החזרה (Return Value):</u>
בניגוד לקריאות מערכת רגילות ב-C שמחזירות 1- ומעדכנות את errno, פונקציות pthread מחזירות את קוד השגיאה עצמו (מספר חיובי) אם הן נכשלות, או 0 אם הן מצליחות.

<u>לוגיקה כוללת של התכנית</u>

הפיצול: התוכנית מתחילה כחוט בודד. הקריאה ל-`pthread_create` יוצרת חוט שני שרץ במקביל.

שיתוף: כפי שתראו בהדפסות, ה-PID (מזהה התהליך) יהיה זהה בשניהם כי הם חולקים את אותם משאבי מערכת, אך ה-TID יהיה שונה.

סנכרון בסיסי: השורה `sleep(2)` <u>הכרחית</u>.
ב-Linux, אם ה-Main Thread מסיים את פונקציית ה-`main`, הוא 'גורר איתו' את כל התהליך לסגירה.
ה-`sleep` מבטיח שהחוט החדש יספיק לרוץ ולהדפיס לפני שהתהליך כולו מת.

© יצחק נודלר

דוגמה המדגימה Race Condition על המשתנה הגלובלי

מטרה: כדי להמחיש את הסכנות בעבודה עם חוטים

המונה המשותף (הדגמה בסיסית)

בדוגמה זו, שני חוטים מנסים לקדם (increment) משתנה גלובלי אחד מיליון פעמים כל אחד.
התוצאה שאנו מצפים 'לראות' בסוף הריצה היא של 2,000,000, אך בפועל:
- תוצאה נמוכה יותר
- ולא רק זאת, אלא גם ושונה בכל הרצה.

```
#include <pthread.h>
#include <stdio.h>

/* Shared global variable */
int counter = 0;

/* Function to increment the counter 1,000,000 times */
void* increment_task(void* arg)
{
    for (int i = 0; i < 1000000; i++) {
        /* The Race Condition happens here:
           This operation is not atomic at the CPU level. */
        counter++;
    }
    return NULL;
}

int main()
{
    pthread_t t1, t2;

    /* Create two threads performing the same task */
    pthread_create(&t1, NULL, increment_task, NULL);
    pthread_create(&t2, NULL, increment_task, NULL);

    /* Wait for both threads to finish execution */
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    /* Print the final result */
    printf("Final Counter Value: %d\n", counter);
    printf("Expected Value:      2000000\n");

    return 0;
}
```


© יצחק נודלר

הסבר לוגי מפורט

הסיבה שהקוד נכשל היא מכיוון שפעולת ה-++ אינה **אטומית** (Atomic) !!
ברמת ה-Assembly של המעבד, הפקודה counter++ מתורגמת לשלושה שלבים נפרדים:

1. **Read**: קריאת הערך מהזיכרון (MM) לתוך אוגר (Register) במעבד.
2. **Modify**: הוספת 1 לערך בתוך האוגר.
3. **Write**: כתיבת הערך המעודכן מהאוגר חזרה לזיכרון.

הבעיה: כאשר שני חוטים רצים במקביל, ייתכן מצב של "שזירה" (Interleaving). חוט א' קורא את הערך 100. באותו רגע, חוט ב' גם קורא את הערך 100. שניהם מעלים אותו ל-101 באוגרים הנפרדים שלהם, ושניהם כותבים 101 חזרה לזיכרון. במקום ששני קידומים יביאו אותנו ל-102, איבדנו קידום אחד בגלל ה-"מרוץ" ביניהם.

דוגמה 2: ניהול חשבון בנק (דוגמה מורכבת ומשמעותית)

כאן נדגים מערכת המבצעת אלפי פעולות של הפקדה (Deposit) ומשיכה (Withdraw) במקביל על ידי חוטים שונים.

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

/* Structure representing a bank account */
typedef struct {
    double balance;
} BankAccount;

BankAccount my_account = {1000.0}; // Starting with $1000

/* Function to simulate many small deposits */
void* deposit_service(void* arg)
{
    for (int i = 0; i < 100000; i++) {
        double temp = my_account.balance;
        temp += 10.0; // Deposit $10
        /* Simulating a tiny delay to increase the chance of a
context switch */
        my_account.balance = temp;
    }
    return NULL;
}

/* Function to simulate many small withdrawals */
void* withdraw_service(void* arg)
{
    for (int i = 0; i < 100000; i++) {
        double temp = my_account.balance;
```


© יצחק נודלר

```
        temp -= 10.0; // Withdraw $10
        my_account.balance = temp;
    }
    return NULL;
}

int main()
{
    pthread_t t1, t2, t3, t4;

    printf("Starting Balance: $%.2f\n", my_account.balance);

    /* Creating multiple threads for deposits and withdrawals */
    pthread_create(&t1, NULL, deposit_service, NULL);
    pthread_create(&t2, NULL, withdraw_service, NULL);
    pthread_create(&t3, NULL, deposit_service, NULL);
    pthread_create(&t4, NULL, withdraw_service, NULL);

    /* Wait for all banking transactions to complete */
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    pthread_join(t4, NULL);

    /* Theoretically, balance should still be $1000.00 */
    printf("Final Balance:    $%.2f\n", my_account.balance);

    return 0;
}
```

הסבר לוגי מפורט

דוגמה זו מעלה את רמת המורכבות בכמה אופנים:

ריבוי חוטים: השתמשנו בארבעה חוטים (שניים מפקידים ושניים מושכים). זה מגדיל משמעותית את כמות ה-"התנגשויות" בזיכרון.

מניפולציה מפורשת: בניגוד ל-++ הפשוט, כאן הפרדנו את הקריאה מהכתיבה על ידי שימוש במשתנה זמני (temp). זה מדמה קוד לוגי אמיתי שבו מבצעים חישובים לפני עדכון הנתונים.

חוסר עקביות (Non-determinism): אם נריץ את התוכנית הזו שלוש פעמים, נקבל שלוש תוצאות שונות לחלוטין. פעם אחת היתרה תהיה $850, פעם אחרת $1120, ולעיתים רחוקות (במקרה) $1000.

הסכנה: צריך להבין שה-Kernel יכול להחליט להפסיק חוט אחד (Context Switch) בדיוק בין השורה שבה הוא חישב את ה-temp לבין השורה שבה הוא עדכן את ה-balance. בזמן שהחוט הראשון "ישן", חוטים אחרים שינו את היתרה בזיכרון. כשהחוט הראשון מתעורר, הוא דורס את השינויים של חבריו עם הערך הישן שהיה לו ב-temp.

© יצחק נודלר

הפתרון לבעיות אלו באמצעות מנגנון ה-Mutex (Mutual Exclusion)

מנגנון זה מבטיח שרק חוט אחד יוכל לגשת לקטע קוד מסוים (הקטע הקריטי) בכל רגע נתון.
נשתמש בדוגמה המשמעותית של חשבון הבנק כדי להראות כיצד "נועלים" את המשאב המשותף.

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

/* Structure representing a bank account with its own lock */
typedef struct {
    double balance;
    pthread_mutex_t lock; // The Mutex: A synchronization object
} BankAccount;

/* Initialize the account with $1000 and an initialized mutex */
BankAccount my_account = {
    .balance = 1000.0,
    .lock = PTHREAD_MUTEX_INITIALIZER // Static initialization of
the mutex
};

/* Function to simulate many small deposits - NOW THREAD SAFE */
void* deposit_service(void* arg)
{
    for (int i = 0; i < 100000; i++) {
        // [CRITICAL SECTION START]
        // Request exclusive access. If another thread has the
lock, this thread waits.
        pthread_mutex_lock(&my_account.lock);

        double temp = my_account.balance;
        temp += 10.0;
        my_account.balance = temp;

        // [CRITICAL SECTION END]
        // Release the lock so other threads can proceed.
        pthread_mutex_unlock(&my_account.lock);
    }
    return NULL;
}

/* Function to simulate many small withdrawals - NOW THREAD SAFE
*/
void* withdraw_service(void* arg)
{
    for (int i = 0; i < 100000; i++) {
        // Attempt to lock. Only one thread (deposit or withdraw)
can hold it.
```


© יצחק נודלר

```
        pthread_mutex_lock(&my_account.lock);

        double temp = my_account.balance;
        temp -= 10.0;
        my_account.balance = temp;

        // Release the lock.
        pthread_mutex_unlock(&my_account.lock);
    }
    return NULL;
}

int main()
{
    pthread_t t1, t2, t3, t4;

    printf("Starting Balance: $%.2f\n", my_account.balance);

    pthread_create(&t1, NULL, deposit_service, NULL);
    pthread_create(&t2, NULL, withdraw_service, NULL);
    pthread_create(&t3, NULL, deposit_service, NULL);
    pthread_create(&t4, NULL, withdraw_service, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    pthread_join(t4, NULL);

    /* With Mutex, the result will ALWAYS be exactly $1000.00 */
    printf("Final Balance:    $%.2f\n", my_account.balance);

    // Cleanup: Destroy the mutex when no longer needed
    pthread_mutex_destroy(&my_account.lock);

    return 0;
}
```

הסבר מפורט: מנגנון ה-Mutex

כדי להבין איך הקוד הזה פותר את הבעיה, עלינו לצלול לתוך הקשר שבין ה-C Library לבין ה-Kernel.

מהו ה-Mutex?

- ה- Mutual Exclusion (Mutex) הוא אובייקט סנכרון שמתנהג כמו "מפתח" יחיד לחדר.

- כאשר חוט רוצה להיכנס ל**קטע הקריטי** (Critical Section) – אותו אזור בקוד שבו ניגשים למשאב משותף כמו ה-`balance` – הוא חייב לקחת את המפתח באמצעות `pthread_mutex_lock`.

- אם המפתח כבר תפוס על ידי חוט אחר, החוט המבקש **נחסם** (Blocked).

© יצחק גוזלר

הלוגיקה מנקודת המבט של ה-Kernel

1. ניסיון הנעילה: כשהחוט קורא ל-`pthread_mutex_lock`, הספרייה בודקת (לרוב בעזרת הוראת מעבד אטומית כמו `Atomic Compare-and-Swap`) אם המנעול פנוי.

2. המתנה (Wait State): אם המנעול תפוס, החוט לא מבצע "לולאה אינסופית" (זה היה מבזבז 'סתם' זמן CPU).
במקום זאת, מתבצעת קריאת מערכת (כמו `futex` ב-Linux) שמעבירה את החוט למצב **Sleep/Blocked**.
ה-Kernel מוציא את החוט מתור התזמון (Scheduler) ולא נותן לו זמן מעבד עד שהמנעול משתחרר.

3. השחרור: כשהחוט המחזיק במנעול קורא ל-`pthread_mutex_unlock`, הוא "מעיר" את אחד החוטים הממתינים בתור (אם יש כאלו).
ה-Kernel מחזיר את החוט שהתעורר למצב `Ready`, והוא יוכל להמשיך בביצוע ברגע שהמתזמן יבחר בו.

ארגומנטים ופונקציות מרכזיות

- **`pthread_mutex_t`:** זהו טיפוס הנתונים של המנעול. הוא מכיל מבנה נתונים פנימי שעוקב אחרי המצב (נעול/פתוח) ואחרי רשימת החוטים הממתינים.

- **`PTHREAD_MUTEX_INITIALIZER`:** זוהי מאקרו לאתחול סטטי. היא קובעת את ערכי ברירת המחדל של המנעול מבלי צורך לקרוא לפונקציית אתחול מפורשת.

- **`pthread_mutex_lock(mutex)`:**
ארגומנט: מצביע ל-mutex.
פעולה: ניסיון לקבל בעלות על המנעול. פעולה זו היא **חוסמת** (Blocking).

- **`pthread_mutex_unlock(mutex)`:**
ארגומנט: מצביע ל-mutex.
פעולה: שחרור המנעול. זוהי פעולה שחייבת להתבצע תמיד על ידי אותו חוט שביצע את הנעילה.

סיכום הלוגיקה של התכנית המשופרת

בזכות ה-Mutex, המצב שבו שני חוטים מעדכנים את ה-`balance` בו-זמנית הפך לבלתי אפשרי. גם אם ה-Kernel מבצע Context Switch באמצע הפעולה של חוט א', חוט ב' שינסה לרוץ ייעצר מיד בשורת ה-`lock` ולא יוכל לגעת ב-`balance` עד שחוט א' יסיים את עבודתו וישחרר את הנעילה. התוצאה הסופית תהיה תמיד עקבית ומדויקת ($1000.00).

© יצחק גוזלר